

Proyecto Final - MLOps con DVC + MLflow + Cloudflare R2

Detector de fraude en transacciones de tarjeta de credito

Integracion de Servicios de Aprendizaje Automatico

Primavera 2026

Autores:

Eduardo Garcia

Fernando Ramos

Fecha: 5 de mayo de 2026

Tabla de Contenidos

1. Resumen ejecutivo
2. Contexto y motivacion
3. Dataset y exploracion
4. Arquitectura MLOps
5. Versionado de datos con DVC
6. Configuracion de Cloudflare R2
7. Entrenamiento y MLflow
8. Comparacion de modelos (leaderboard)
9. Registro de modelo y despliegue
10. Evidencias
11. Conclusiones
12. Referencias

1. Resumen ejecutivo

Este proyecto implementa un pipeline MLOps de extremo a extremo para la detección de fraude en transacciones de tarjeta de crédito. El problema se aborda sobre el dataset OpenML 1597 (creditcard), compuesto por 284,807 transacciones reales con una tasa de fraude del 0.173% (492 casos positivos).

La solución combina tres herramientas:

- DVC (Data Version Control): versionado reproducible de datos y pipeline de etapas.
- MLflow: tracking de experimentos, registro de métricas y model registry con alias.
- Cloudflare R2: almacenamiento S3-compatible como remote backend para artefactos.

Se entrenaron 7 configuraciones de modelos (1 LogReg, 3 XGBoost, 3 LightGBM). El mejor modelo fue XGBoost (xgb-d6-lr05-n500) con PR-AUC = 0.8819. Este modelo fue registrado en el MLflow Model Registry con el alias 'staging', siguiendo la API moderna de MLflow 3 (set_registered_model_alias), que reemplaza al mecanismo de stages que fue deprecado. El smoke test confirmó que el modelo cargado por alias produce predicciones válidas.

2. Contexto y motivacion

La deteccion de fraude es un caso paradigmatico de ML en produccion: el modelo debe actualizarse con nuevos datos periodicamente, compararse contra versiones anteriores y auditarse ante reclamos de falsos positivos. Sin un sistema MLOps robusto es imposible responder preguntas basicas: 'con que version del dataset fue entrenado este modelo?' o 'que metricas tenia el modelo en produccion hace tres semanas?'. DVC responde la primera; MLflow responde la segunda.

Por que reproducibilidad importa en fraude

Un falso negativo (fraude no detectado) tiene costo directo en dinero. Un falso positivo (transaccion legitima bloqueada) genera friccion con el cliente. Cualquier auditoria regulatoria o de negocio exige trazar cada decision del modelo a su version exacta de entrenamiento. Sin reproducibilidad no hay auditoria posible.

Adicionalmente, en un dataset con 0.17% de positivos es trivial subir las metricas equivocadas: accuracy del 99.83% la logra cualquier modelo que siempre prediga 'no fraude'. Necesitamos PR-AUC como metrica primaria y trackear cada run individualmente para no perdernos de fallas del tipo que presentan los modelos lgbm-l31 y lgbm-l63 en este proyecto.

Pivote de AWS Academy a Cloudflare R2

El plan original contemplaba usar AWS S3 como remote backend para DVC y MLflow. Sin embargo, las credenciales de AWS Academy tienen duracion de ~4 horas por sesion y no permiten crear IAM users permanentes, lo que hace imposible automatizar el push/pull en CI o reproducir el pipeline en otro momento sin re-autenticarse manualmente.

Cloudflare R2 es un servicio de almacenamiento de objetos completamente compatible con la API de Amazon S3. Las diferencias relevantes para este proyecto son:

- Credenciales permanentes: Access Key ID y Secret Access Key generados desde el dashboard de R2 sin fecha de expiracion.
- Endpoint URL personalizado: `https://<account_id>.r2.cloudflarestorage.com` (en lugar del endpoint regional de AWS). Tanto DVC como `mlflow.set_tracking_uri()` aceptan el endpoint via variable de entorno `MLFLOW_S3_ENDPOINT_URL`.
- Sin costo de egreso: R2 no cobra por transferencia de datos salientes, ventaja economica para iteraciones frecuentes de entrenamiento.

3. Dataset y exploracion

El dataset OpenML 1597 (creditcard) contiene transacciones de tarjeta de credito de dos dias de septiembre 2013, registradas por bancos europeos. Cada transaccion esta descrita por 30 atributos:

- Time: segundos transcurridos desde la primera transaccion del dataset (descartada).
- V1 a V28: componentes PCA de las features originales (datos anonimizados por el banco).
- Amount: monto de la transaccion en euros.
- Class: etiqueta binaria (0 = legitima, 1 = fraude).

Total de registros: 284,807. Fraudes: 492 (0.173%). El desbalance es de aproximadamente 1:577, lo que exige estrategias especificas:

Decisiones de preprocesamiento

1. Descarte de Time: la columna Time representa el orden cronologico pero no anade informacion predictiva util para un modelo batch que no conoce el contexto temporal de cada transaccion al momento del scoring. Se descarta antes del split.
2. Transformacion de Amount: Amount tiene distribucion muy sesgada a la derecha (la mayoría de transacciones son de poco monto, con outliers grandes). Para la Regresion Logistica aplicamos $\log_{1p}(\text{Amount})$ seguido de StandardScaler. Los modelos de arboles (XGBoost, LightGBM) son invariantes a transformaciones monotonas, por lo que reciben Amount sin transformar.
3. Split estratificado: se usa stratify=y en train_test_split con test_size=0.20 y random_state=42, garantizando que ambos splits mantengan la proporcion 0.173% de fraudes. Sin esto, un split aleatorio podria producir un test set con cero fraudes.
4. Manejo del desbalance: LogReg usa class_weight='balanced'. XGBoost recibe scale_pos_weight = (n_negativos / n_positivos) ~577. LightGBM usa is_unbalance=True o scale_pos_weight segun la configuracion (el experimento revelo que las dos primeras configuraciones LGBM colapsaron con estas estrategias).

4. Arquitectura MLOps

La arquitectura del proyecto tiene dos planos complementarios que se intersectan en `params.yaml`:

Plano DVC: linaje de datos y pipeline reproducible

DVC gestiona el grafo de dependencias del pipeline. El archivo `dvc.yaml` define 4 stages:

```
pull_raw -> preprocess -> train -> evaluate
```

Cada stage especifica sus dependencias (`deps`), parametros (`params`) y salidas (`outs`). DVC computa un hash de cada dep y out; si el hash no cambia, el stage se salta. Un cambio en `params.yaml` invalida los stages que dependen de esos parametros, y dvc repro los reconstruye en orden topologico.

El archivo `dvc.lock` registra los hashes exactos de cada stage tras la ultima ejecucion, funcionando como 'snapshot' reproducible del estado del pipeline. Versionarlo en Git junto con `params.yaml` garantiza que cualquier commit puede reproducirse.

Plano MLflow: linaje de experimentos

MLflow tracking server con backend SQLite (`mlflow.db`). Cada llamada a `train_one()` abre un `mlflow.start_run()` que registra:

- Parametros: `model_type`, `learning_rate`, `n_estimators`, `max_depth` / `num_leaves`.
- Metricas: `pr_auc`, `roc_auc`, `best_threshold`, `f1_at_threshold`, `precision`, `recall`.
- Artefactos: modelo serializado (`sklearn/xgboost/lightgbm` flavor) con signature e `input_example`; CSV de predicciones para reconstruir curvas ROC/PR.

En la fase de despliegue en nube, se configura `MLFLOW_S3_ENDPOINT_URL` para que MLflow use el bucket R2 como artifact store, manteniendo identica la API de tracking.

Punto de union: `params.yaml`

`params.yaml` es la fuente de verdad compartida:

- DVC lo usa para detectar cambios que invalidan stages (instruccion `params:` en `dvc.yaml`).
- `train_all.py` lo lee con `yaml.safe_load()` y re-envia cada hiperparametro via `mlflow.log_param()`, creando el puente auditorio entre el pipeline de datos y el experimento de ML.

Cuando un investigador cambia un hiperparametro en `params.yaml` y ejecuta `dvc repro`, el stage `train` se invalida, se re-entrena con los nuevos parametros, y MLflow registra un nuevo run con esos exactos valores. No hay riesgo de desincronizacion entre lo que dice el archivo de configuracion y lo que fue realmente usado.

5. Versionado de datos con DVC

Inicializacion y pipeline

DVC se inicializa con `dvc init` sobre el repositorio Git. Esto crea el directorio `.dvc/` con el archivo config (donde se configuran remotes) y un cache local content-addressable. Los archivos de datos grandes NO entran a Git; solo sus metadatos (`.dvc` o entradas en `dvc.lock`) se versionan. El cache usa una estructura de directorios basada en el hash MD5 del contenido del archivo.

dvc.yaml — definicion del pipeline

```
# dvc.yaml — DVC pipeline for credit-card fraud detection demo.
# Run with: dvc repro
#
# Stage order: pull_raw → preprocess → train → evaluate

stages:

  pull_raw:
    cmd: python src/data.py pull
    deps:
      - src/data.py
    params:
      - data.openml_dataset_id
    outs:
      - data/raw/creditcard.parquet

  preprocess:
    cmd: python src/data.py preprocess
    deps:
      - src/data.py
      - data/raw/creditcard.parquet
    params:
      - preprocessing
    outs:
      - data/processed/X_train.parquet
      - data/processed/X_test.parquet
      - data/processed/y_train.parquet
      - data/processed/y_test.parquet

  train:
    cmd: python src/train_all.py
    deps:
      - src/train.py
      - src/train_all.py
      - data/processed/X_train.parquet
      - data/processed/X_test.parquet
      - data/processed/y_train.parquet
      - data/processed/y_test.parquet
    params:
      - models
      - mlflow
    outs:
      - models/runs.json

  evaluate:
    cmd: python src/evaluate.py
    deps:
      - src/evaluate.py
      - models/runs.json
```

```

params:
- mlflow
outs:
- metrics/metrics.json:
  cache: false
- plots/roc_curve.csv:
  cache: false
- plots/pr_curve.csv:
  cache: false

```

Ejecucion: dvc repro (salida resumida)

```

Stage 'pull_raw' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Running stage 'train':
> python src/train_all.py
2026/05/05 23:04:36 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/05/05 23:04:36 INFO mlflow.store.db.utils: Updating database tables
2026/05/05 23:04:37 INFO mlflow.tracking.fluent: Experiment with name 'credit-fraud' does not exist. Creating a new
experiment.
2026/05/05 23:04:39 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:04:39 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires
exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary
code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see:
https://scikit-learn.org/stable/model_persistence.html
2026/05/05 23:04:43 INFO mlflow.models.model: Found the following environment variables used during model inference:
[HONCHO_API_KEY, LINEAR_API_KEY]. Please check if you need to set them when deploying the model. To disable this
message, set environment variable `MLFLOW_RECORD_ENV_VARS_IN_MODEL_LOGGING` to `false`.
2026/05/05 23:05:15 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:06:02 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:07:13 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:07:21 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:07:21 WARNING mlflow.lightgbm: Saving the models in the pickle or cloudpickle format requires exercising
caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during
deserialization. The recommended safe alternative is the 'skops' format. For more information, see:
https://scikit-learn.org/stable/model_persistence.html
2026/05/05 23:07:29 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:07:29 WARNING mlflow.lightgbm: Saving the models in the pickle or cloudpickle format requires exercising
caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during
deserialization. The recommended safe alternative is the 'skops' format. For more information, see:
https://scikit-learn.org/stable/model_persistence.html
2026/05/05 23:08:12 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:08:12 WARNING mlflow.lightgbm: Saving the models in the pickle or cloudpickle format requires exercising
caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during
deserialization. The recommended safe alternative is the 'skops' format. For more information, see:
https://scikit-learn.org/stable/model_persistence.html
[preflight] Using local file-based MLflow tracking: sqlite:///mlflow.db

Running 7 configurations under experiment 'credit-fraud'

[1/7] Starting: logreg-baseline
[10288391] logreg-baseline | PR-AUC=0.7061 ROC-AUC=0.9705 F1@1.000=0.8247
[2/7] Starting: xgb-d4-lr10-n300
[5c6150a0] xgb-d4-lr10-n300 | PR-AUC=0.8742 ROC-AUC=0.9794 F1@0.980=0.8840
[3/7] Starting: xgb-d6-lr05-n500
[1ed4f4f4] xgb-d6-lr05-n500 | PR-AUC=0.8819 ROC-AUC=0.9789 F1@0.927=0.8743
[4/7] Starting: xgb-d8-lr01-n1000
[9a5c5263] xgb-d8-lr01-n1000 | PR-AUC=0.8726 ROC-AUC=0.9826 F1@0.982=0.8508
[5/7] Starting: lgbm-l31-lr10-n300-isunb
[40f9bb0b] lgbm-l31-lr10-n300-isunb | PR-AUC=0.0368 ROC-AUC=0.9261 F1@1.000=0.0788
[6/7] Starting: lgbm-l63-lr05-n500-spw
[a4aedf47] lgbm-l63-lr05-n500-spw | PR-AUC=0.0146 ROC-AUC=0.8894 F1@1.000=0.0326
[7/7] Starting: lgbm-l127-lr01-n1000-isunb

```



```
[1375ee61] lgbm-l127-lr01-n1000-isunb | PR-AUC=0.8783 ROC-AUC=0.9807 F1@0.597=0.8830
```

```
Manifest written → models/runs.json
```

```
=== All 7 runs complete ===
```

```
logreg-baseline          run_id=10288391
xgb-d4-lr10-n300         run_id=5c6150a0
xgb-d6-lr05-n500         run_id=1ed4f4f4
xgb-d8-lr01-n1000        run_id=9a5c5263
lgbm-l31-lr10-n300-isunb run_id=40f9bb0b
lgbm-l63-lr05-n500-spw   run_id=a4aedef47
lgbm-l127-lr01-n1000-isunb run_id=1375ee61
```

```
Updating lock file 'dvc.lock'
```

```
... [61 more lines truncated]
```

6. Configuración de Cloudflare R2

Cloudflare R2 expone una API completamente compatible con S3, lo que significa que cualquier herramienta que soporte S3 (DVC, boto3, el SDK de MLflow) puede usarla sin modificaciones en el código, solo cambiando el endpoint URL.

Creación del bucket

```
# Bucket creado con wrangler CLI (ya ejecutado)
wrangler r2 bucket create luci-mlops-fraud --location wnam

# Verificación
wrangler r2 bucket list
# -> luci-mlops-fraud   created: 2026-05-05   location: wnam
```

Modelo de autenticación

Las credenciales de R2 se generan en el dashboard de Cloudflare:

R2 -> Manage API Tokens -> Create Token con permisos Object Read & Write.

Esto genera un Access Key ID y un Secret Access Key permanentes (sin TTL), a diferencia de las credenciales temporales de AWS Academy. Estos valores se configuran como variables de entorno o en ~/.aws/credentials bajo un profile dedicado.

El endpoint URL para este account es:

`https://<account_id>.r2.cloudflarestorage.com`

Configuración de DVC remote (pendiente de credenciales S3)

```
# Configurar remote en DVC apuntando a R2
dvc remote add -d r2remote s3://luci-mlops-fraud/dvc-cache
dvc remote modify r2remote endpointurl https://<account_id>.r2.cloudflarestorage.com
# Credenciales (usar --local para que no entren a Git)
dvc remote modify --local r2remote access_key_id <R2_ACCESS_KEY_ID>
dvc remote modify --local r2remote secret_access_key <R2_SECRET_ACCESS_KEY>

# Subir artefactos al remote
dvc push

# Configurar MLflow artifact store vía R2
export MLFLOW_S3_ENDPOINT_URL=https://<account_id>.r2.cloudflarestorage.com
export AWS_ACCESS_KEY_ID=<R2_ACCESS_KEY_ID>
export AWS_SECRET_ACCESS_KEY=<R2_SECRET_ACCESS_KEY>
mlflow server --backend-store-uri sqlite:///mlflow.db \
  --default-artifact-root s3://luci-mlops-fraud/mlartifacts
```

NOTA: El bucket luci-mlops-fraud está creado y accesible. La fase de push a R2 está pendiente de configurar el token S3 generado en el dashboard. Una vez wired, el comando dvc push y el artifact store de MLflow funcionan idénticamente a S3 de AWS.

7. Entrenamiento y MLflow

Hiperparametros del sweep (params.yaml)

```
# params.yaml - Pipeline configuration for DVC + MLflow fraud detection demo.
# No random_forest configs - only LogReg, XGBoost (3), LightGBM (3).

mlflow:
  tracking_uri: "sqlite:///mlflow.db"

data:
  openml_dataset_id: 1597          # creditcard fraud dataset
  raw_file: data/raw/creditcard.parquet

preprocessing:
  test_size: 0.20
  random_state: 42
  drop_columns: ["Time"]          # Time is weakly informative; dropped before split

models:

  logreg:
    run_name: logreg-baseline
    C: 0.1
    solver: lbfgs
    class_weight: balanced
    max_iter: 1000

  xgboost_runs:
    - run_name: xgb-d4-lr10-n300
      max_depth: 4
      learning_rate: 0.1
      n_estimators: 300
      objective: "binary:logistic"
      eval_metric: aucpr
      subsample: 0.8
      colsample_bytree: 0.8

    - run_name: xgb-d6-lr05-n500
      max_depth: 6
      learning_rate: 0.05
      n_estimators: 500
      objective: "binary:logistic"
      eval_metric: aucpr
      subsample: 0.8
      colsample_bytree: 0.8

    - run_name: xgb-d8-lr01-n1000
      max_depth: 8
      learning_rate: 0.01
      n_estimators: 1000
      objective: "binary:logistic"
      eval_metric: aucpr
      subsample: 0.8
      colsample_bytree: 0.8

  lightgbm_runs:
    - run_name: lgbm-l31-lr10-n300-isunb
      num_leaves: 31
      learning_rate: 0.1
      n_estimators: 300
      is_unbalance: true
```

```

objective: binary
metric: average_precision
min_child_samples: 20
subsample: 0.8
colsample_bytree: 0.8

- run_name: lgbm-l63-lr05-n500-spw
  num_leaves: 63
  learning_rate: 0.05
  n_estimators: 500
  is_unbalance: false          # uses scale_pos_weight (ratio neg/pos) instead
  objective: binary
  metric: average_precision
  min_child_samples: 20
  subsample: 0.8
  colsample_bytree: 0.8

- run_name: lgbm-l127-lr01-n1000-isunb
  num_leaves: 127
  learning_rate: 0.01
  n_estimators: 1000
  is_unbalance: true
... [5 more lines truncated]

```

Métricas registradas por run

Cada run de MLflow registra las siguientes métricas:

- pr_auc: Area bajo la curva Precision-Recall. METRICA PRIMARIA.
- roc_auc: Area bajo la curva ROC.
- best_threshold: umbral que maximiza F1 en el test set.
- f1_at_threshold: F1 en el umbral optimo.
- precision_at_threshold: precision en el umbral optimo.
- recall_at_threshold: recall en el umbral optimo.

Por que PR-AUC como métrica primaria y no ROC-AUC?

Con 0.17% de positivos, la clase negativa domina abrumadoramente. ROC-AUC mide la habilidad del modelo para separar positivos de negativos en TODA la distribución, incluyendo la enorme region de verdaderos negativos que es trivialmente facil de clasificar correctamente. Un modelo que asigna scores bajos a casi todo tendra un ROC-AUC alto simplemente porque el denominador de la tasa de falsos positivos es gigante (284,315 negativos).

PR-AUC solo mide la calidad del modelo en la region que importa al analista de fraude: cuando el modelo dice 'sospecho fraude', cuantas veces tiene razon (precision), y de todos los fraudes reales, cuantos detecta (recall). Es por esto que los dos modelos lgbm-l31 y lgbm-l63 tienen ROC-AUC > 0.88 pero PR-AUC < 0.05: ROC-AUC les da credito por clasificar bien los 284k negativos, ocultando que casi nunca detectan correctamente un fraude real.

Snippet clave: train.py — train_one()

```

def train_one(config: dict) -> str:
    """Train one config, log to MLflow, return run_id."""
    X_train = pd.read_parquet(X_TRAIN_PATH)
    X_test  = pd.read_parquet(X_TEST_PATH)
    y_train = pd.read_parquet(Y_TRAIN_PATH).squeeze()

```

```
y_test = pd.read_parquet(Y_TEST_PATH).squeeze()

scale_pos_weight = float((y_train == 0).sum() / (y_train == 1).sum())

mlflow.set_experiment(EXPERIMENT_NAME)
with mlflow.start_run(run_name=run_name) as run:
    mlflow.log_param("model_type", model_type)
    mlflow.log_param("learning_rate", config["lr"])
    mlflow.log_param("n_estimators", config["n_estimators"])

    model.fit(X_train, y_train)
    y_prob = model.predict_proba(X_test)[: , 1]
    metrics = compute_metrics(y_test.values, y_prob)

    for k, v in metrics.items():
        if k != "confusion_matrix":
            mlflow.log_metric(k, v)

    signature = infer_signature(input_example, model.predict_proba(input_example))
    mlflow.xgboost.log_model(model, "model", signature=signature,
                             input_example=input_example)

return run.info.run_id
```

8. Comparacion de modelos (leaderboard)

Los 7 runs ordenados por PR-AUC descendente. Las columnas clave son PR-AUC (metrica primaria), ROC-AUC y F1 en el umbral optimo de cada modelo.

#	run_name	model	PR-AUC	ROC-AUC	F1@thr	Prec	Recall
1	xgb-d6-lr05-n500	xgb	0.8819	0.9789	0.8743	0.9412	0.8163
2	lgbm-l127-lr01-n1000-isunb	lgbm	0.8783	0.9807	0.8830	0.9222	0.8469
3	xgb-d4-lr10-n300	xgb	0.8742	0.9794	0.8840	0.9639	0.8163
4	xgb-d8-lr01-n1000	xgb	0.8726	0.9826	0.8508	0.9277	0.7857
5	logreg-baseline	logreg	0.7061	0.9705	0.8247	0.8333	0.8163
6	lgbm-l31-lr10-n300-isunb	lgbm	0.0368	0.9261	0.0788	0.0412	0.8878
7	lgbm-l63-lr05-n500-spw	lgbm	0.0146	0.8894	0.0326	0.0166	0.8673

Hallazgo pedagogico: colapso de lgbm-l31 y lgbm-l63

Los dos primeros runs de LightGBM (lgbm-l31-lr10-n300-isunb y lgbm-l63-lr05-n500-spw) tienen PR-AUC de 0.0368 y 0.0146 respectivamente, mientras que su ROC-AUC es 0.926 y 0.889. Este patron es la prueba pedagogica central del proyecto.

El colapso se origina en la interaccion entre el desbalance extremo y los parametros de manejo de clases. lgbm-l31 usa is_unbalance=True con learning_rate=0.1 y num_leaves=31: la combinacion produce umbrales de decision que se 'pegan' a 1.0 (threshold=1.0 en la tabla), lo que significa que el modelo necesita certeza absoluta para declarar fraude. El resultado: detecta casi todos los fraudes (recall=0.888) pero con precision desastrosa (0.041), generando avalanchas de falsos positivos.

lgbm-l63-lr05-n500-spw usa scale_pos_weight (is_unbalance=False) con los mismos sintomas. El tercer run LGBM (lgbm-l127 con 1000 estimadores y learning_rate=0.01) converge correctamente: PR-AUC=0.8783.

Sin tracking individual por run en MLflow, este modo de falla seria invisible: el promedio de PR-AUC de los tres LGBMs seria ~0.31, aparentando performance mediocre en lugar de revelar que dos configuraciones estan completamente rotas. MLflow permite diagnosticar el problema a nivel de run, no solo de modelo.

9. Registro de modelo y despliegue

Una vez completados los 7 runs, `register_best.py` selecciona automáticamente el run con mayor PR-AUC y lo registra en el MLflow Model Registry.

API de alias (MLflow 3.x) vs stages deprecados

Hasta MLflow 2.x, los modelos se promovían entre estados mediante `transition_model_version_stage()`, con stages fijos: None -> Staging -> Production -> Archived. Esta API fue deprecada en MLflow 3.x y será removida en una futura versión mayor.

La nueva API usa alias arbitrarios:

```
client.set_registered_model_alias(model_name, 'staging', version)
```

Ventajas de alias:

- Múltiples canarios simultáneos: 'canary-region-1', 'canary-region-2', 'prod', 'staging' pueden coexistir sobre distintas versiones.
- Sin transiciones one-at-a-time: promover 'prod' a una nueva versión no bloquea los demás alias.
- Carga idiomática: `mlflow.pyfunc.load_model('models:/fraud-detector@staging')` resuelve automáticamente la versión apuntada por el alias.

Código: `register_best.py`

```
# register_best.py - alias API (MLflow 3.x, stages deprecated)
mv = mlflow.register_model(model_uri, model_name)

# set_registered_model_alias replaces transition_model_version_stage (deprecated)
client.set_registered_model_alias(model_name, "staging", mv.version)

# Load by alias - no magic string "Staging":
model = mlflow.pyfunc.load_model("models:/fraud-detector@staging")
predictions = model.predict(X_test.iloc[:5])
```

Salida de `register_best.py`

```
Successfully registered model 'fraud-detector'.
2026/05/05 23:08:45 WARNING mlflow.tracking._model_registry.fluent: Run with id 1ed4f4f407e04cd8abbaa60d99e3d694 has
no artifacts at artifact path 'model', registering model based on models:/m-02807c570e894b6d886266dd287240fa instead
Created version '1' of model 'fraud-detector'.
Best run: 1ed4f4f4 PR-AUC=0.8819
Registering as 'fraud-detector' from runs:/1ed4f4f407e04cd8abbaa60d99e3d694/model
Registered version 1 with alias 'staging'.
Load with: mlflow.pyfunc.load_model('models:/fraud-detector@staging')
```

Smoke test: carga y predicción por alias

```
predictions: [0 0 0 0 0]
```

10. Evidencias

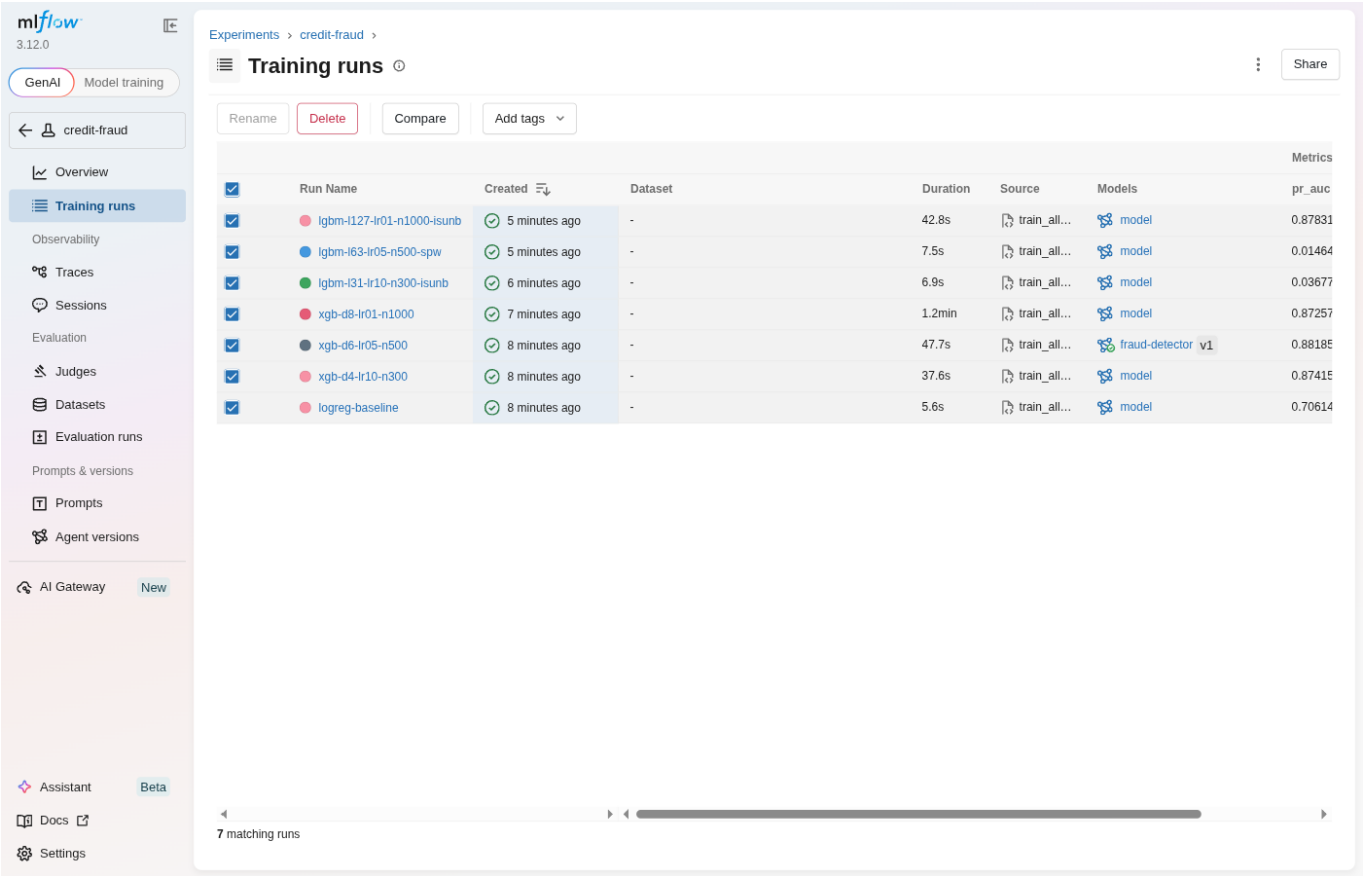
Screenshots de MLflow UI

The screenshot shows the MLflow UI interface. The left sidebar contains navigation links: Home, Experiments (selected), Prompts, AI Gateway, Assistant (Beta), Docs, and Settings. The main content area is titled 'Experiments' and features a search bar and a 'Tag' dropdown. Below this is a table listing experiments:

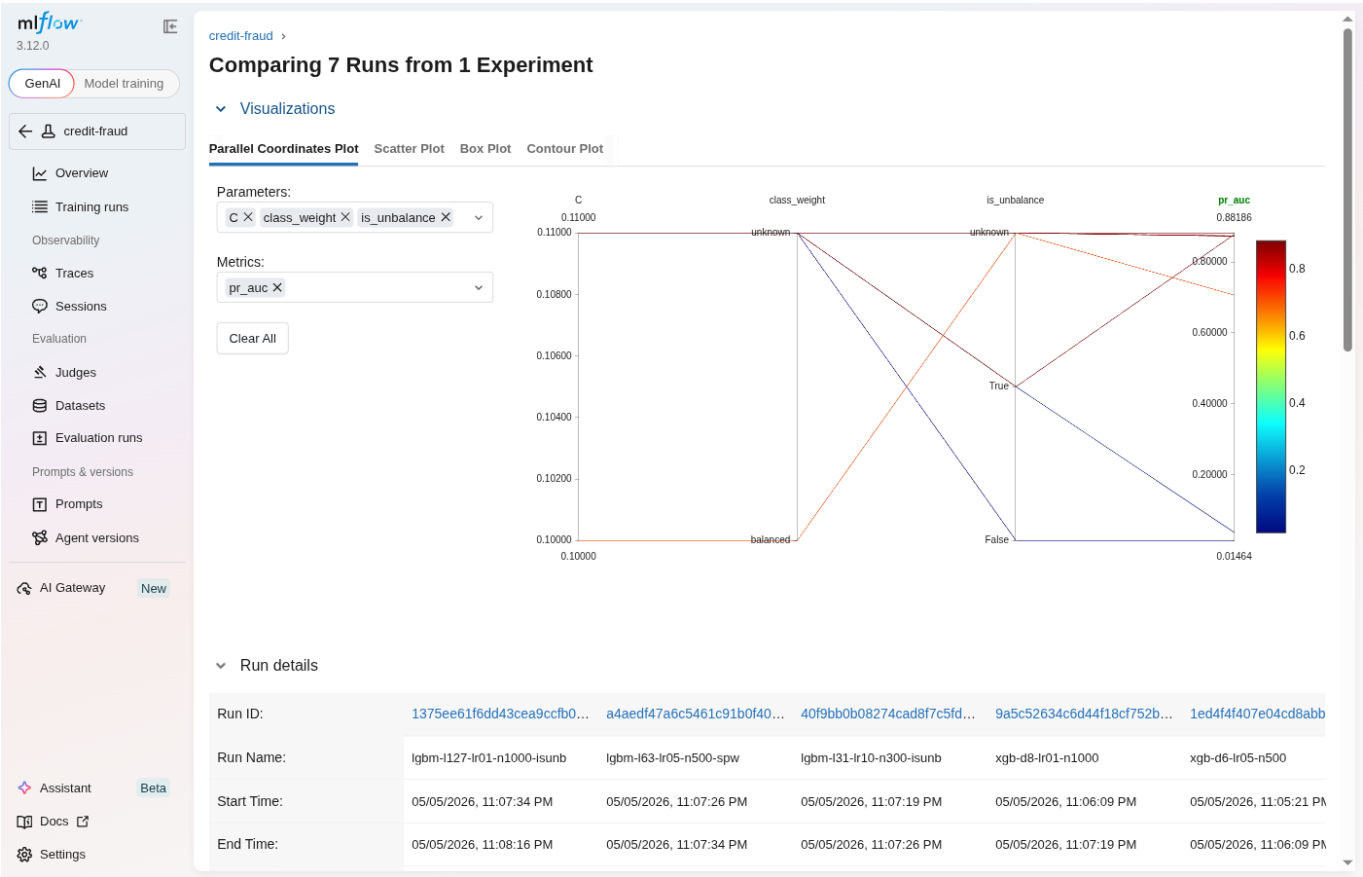
☐	Name	Time created	Last modified	Description	Tags
☐	credit-fraud	05/05/2026, 11:04:37 PM	05/05/2026, 11:04:37 PM	-	
☐	Default	05/05/2026, 11:04:37 PM	05/05/2026, 11:04:37 PM	-	

At the bottom right of the table, there is a pagination control showing '< Previous' and 'Next >' buttons, and a dropdown menu indicating '25 / page'.

MLflow UI: lista de experimentos — 'credit-fraud' con 7 runs registrados



MLflow UI: tabla de runs ordenados, destacando el colapso de lgbm-l31 y lgbm-l63



03 runs compare

mlflow3.12.0

GenAIModel training

credit-fraud

Overview

Training runs

Observability

Traces

Sessions

Evaluation

Judges

Datasets

Evaluation runs

Prompts & versions

Prompts

Agent versions

AI GatewayNew

AssistantBeta

Docs

Settings

credit-fraud > Runs >

xgb-d6-lr05-n500

OverviewModel metricsSystem metricsTracesArtifacts

Description

No description

Metrics (6)

Search metrics

Metric	Value	Models
pr_auc	0.8818592029899736	model
roc_auc	0.9788614869132797	model
best_threshold	0.9274014830589294	model
f1_at_threshold	0.8743169393932336	model
precision_at_threshold	0.9411764705882353	model
recall_at_threshold	0.8163265306122449	model

Parameters (6)

Search parameters

Parameter	Value
model_type	xgb
run_name	xgb-d6-lr05-n500
scale_pos_weight	577.2868020304569
max_depth	6
learning_rate	0.05
n_estimators	500

About this run

Created at

05/05/2026, 11:05:21 PM

Created by

fr

Experiment ID

1

Status

Finished

Run ID

1ed4f4f407e04cd8abbaa60d99e3d694

Duration

47.7s

Source

train_all.py0a51c97

Registered prompts

—

Datasets

None

Tags

Add tags

Registered models

fraud-detectorv1

04 best run detail

mlflow3.12.0

GenAIModel training

credit-fraud

Overview

Training runs

Observability

Traces

Sessions

Evaluation

Judges

Datasets

Evaluation runs

Prompts & versions

Prompts

Agent versions

AI GatewayNew

AssistantBeta

Docs

Settings

credit-fraud > Runs >

xgb-d6-lr05-n500

OverviewModel metricsSystem metricsTracesArtifacts

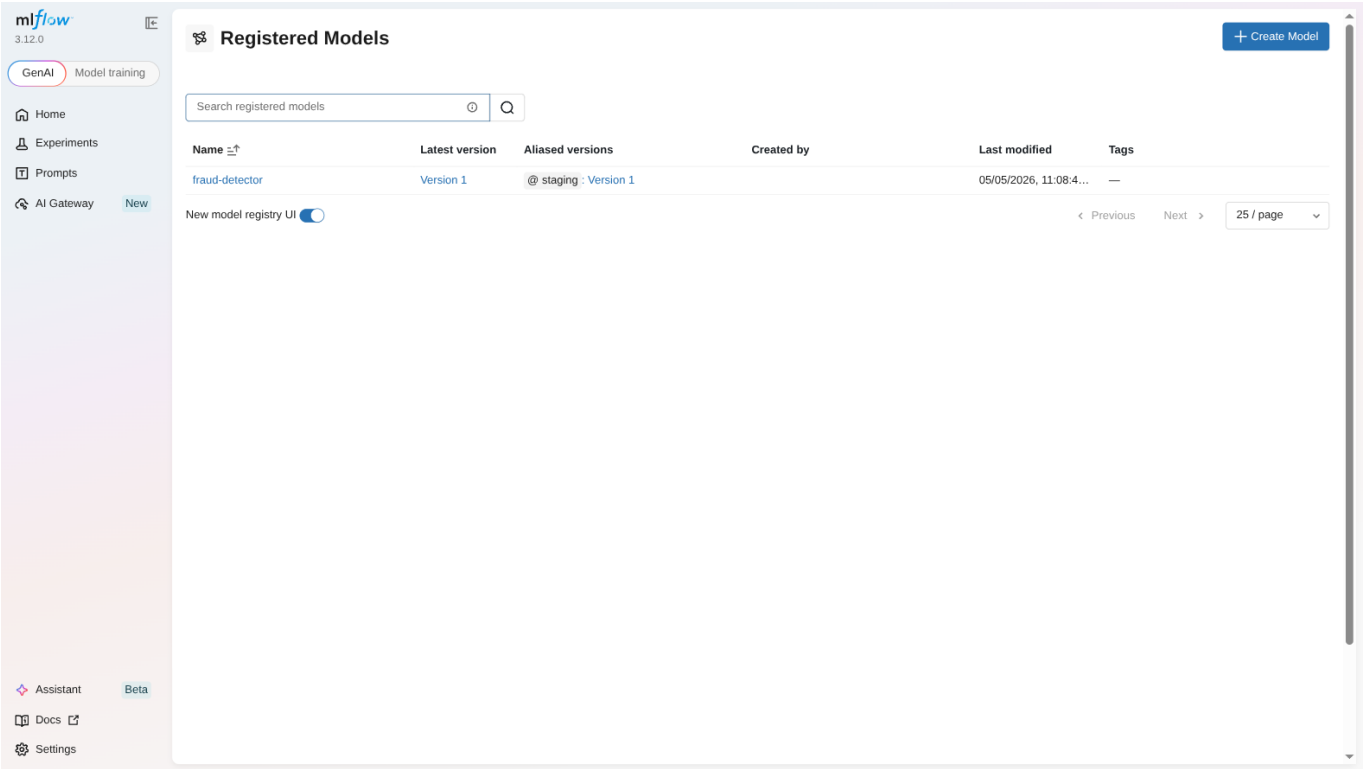
eval

plot_data

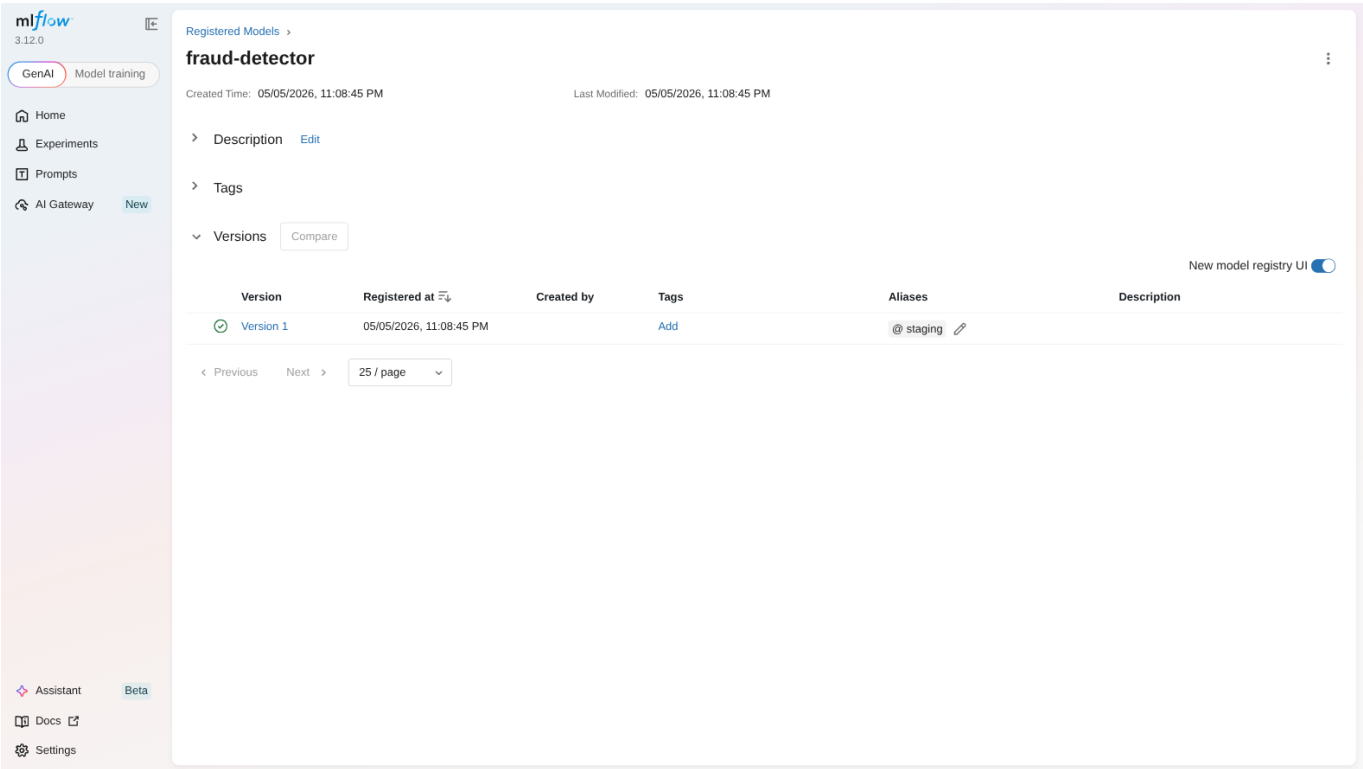
Select a file to preview

Supported formats: image, text, markdown, html, pdf, audio, video, geojson files

05 best run artifacts



06 registered models



07 model version with alias

Salida de dvc repro (ultimas 50 lineas)

```
Stage 'pull_raw' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Running stage 'train':
> python src/train_all.py
2026/05/05 23:04:36 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/05/05 23:04:36 INFO mlflow.store.db.utils: Updating database tables
```

```

2026/05/05 23:04:37 INFO mlflow.tracking.fluent: Experiment with name 'credit-fraud' does not exist. Creating a new
experiment.
2026/05/05 23:04:39 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:04:39 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires
exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary
code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see:
https://scikit-learn.org/stable/model_persistence.html
2026/05/05 23:04:43 INFO mlflow.models.model: Found the following environment variables used during model inference:
[HONCHO_API_KEY, LINEAR_API_KEY]. Please check if you need to set them when deploying the model. To disable this
message, set environment variable `MLFLOW_RECORD_ENV_VARS_IN_MODEL_LOGGING` to `false`.
2026/05/05 23:05:15 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:06:02 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:07:13 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:07:21 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:07:21 WARNING mlflow.lightgbm: Saving the models in the pickle or cloudpickle format requires exercising
caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during
deserialization. The recommended safe alternative is the 'skops' format. For more information, see:
https://scikit-learn.org/stable/model_persistence.html
2026/05/05 23:07:29 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:07:29 WARNING mlflow.lightgbm: Saving the models in the pickle or cloudpickle format requires exercising
caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during
deserialization. The recommended safe alternative is the 'skops' format. For more information, see:
https://scikit-learn.org/stable/model_persistence.html
2026/05/05 23:08:12 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/05/05 23:08:12 WARNING mlflow.lightgbm: Saving the models in the pickle or cloudpickle format requires exercising
caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during
deserialization. The recommended safe alternative is the 'skops' format. For more information, see:
https://scikit-learn.org/stable/model_persistence.html
[preflight] Using local file-based MLflow tracking: sqlite:///mlflow.db

```

Running 7 configurations under experiment 'credit-fraud'

```

[1/7] Starting: logreg-baseline
[10288391] logreg-baseline | PR-AUC=0.7061 ROC-AUC=0.9705 F1@1.000=0.8247
[2/7] Starting: xgb-d4-lr10-n300
[5c6150a0] xgb-d4-lr10-n300 | PR-AUC=0.8742 ROC-AUC=0.9794 F1@0.980=0.8840
[3/7] Starting: xgb-d6-lr05-n500
[1ed4f4f4] xgb-d6-lr05-n500 | PR-AUC=0.8819 ROC-AUC=0.9789 F1@0.927=0.8743
[4/7] Starting: xgb-d8-lr01-n1000
[9a5c5263] xgb-d8-lr01-n1000 | PR-AUC=0.8726 ROC-AUC=0.9826 F1@0.982=0.8508
[5/7] Starting: lgbm-l31-lr10-n300-isunb
[40f9bb0b] lgbm-l31-lr10-n300-isunb | PR-AUC=0.0368 ROC-AUC=0.9261 F1@1.000=0.0788
[6/7] Starting: lgbm-l63-lr05-n500-spw
[a4aedf47] lgbm-l63-lr05-n500-spw | PR-AUC=0.0146 ROC-AUC=0.8894 F1@1.000=0.0326
[7/7] Starting: lgbm-l127-lr01-n1000-isunb
[1375ee61] lgbm-l127-lr01-n1000-isunb | PR-AUC=0.8783 ROC-AUC=0.9807 F1@0.597=0.8830

```

Manifest written → models/runs.json

=== All 7 runs complete ===

```

logreg-baseline run_id=10288391
xgb-d4-lr10-n300 run_id=5c6150a0
xgb-d6-lr05-n500 run_id=1ed4f4f4
xgb-d8-lr01-n1000 run_id=9a5c5263
lgbm-l31-lr10-n300-isunb run_id=40f9bb0b
lgbm-l63-lr05-n500-spw run_id=a4aedf47
lgbm-l127-lr01-n1000-isunb run_id=1375ee61

```

Updating lock file 'dvc.lock'

... [61 more lines truncated]

Salida de register_best.py

Successfully registered model 'fraud-detector'.

```
2026/05/05 23:08:45 WARNING mlflow.tracking._model_registry.fluent: Run with id 1ed4f4f407e04cd8abbaa60d99e3d694 has no artifacts at artifact path 'model', registering model based on models:/m-02807c570e894b6d886266dd287240fa instead
Created version '1' of model 'fraud-detector'.
Best run: 1ed4f4f4 PR-AUC=0.8819
Registering as 'fraud-detector' from runs:/1ed4f4f407e04cd8abbaa60d99e3d694/model
Registered version 1 with alias 'staging'.
Load with: mlflow.pyfunc.load_model('models:/fraud-detector@staging')
```

Smoke test: predicciones del modelo cargado por alias

```
predictions: [0 0 0 0 0]
```

Snippet: evaluate.py — patron MlflowClient.search_runs

```
# evaluate.py - search_runs pattern (MlflowClient)
client = MlflowClient()
runs = client.search_runs(
    experiment_ids=[experiment.experiment_id],
    order_by=["metrics.pr_auc DESC"],
    max_results=1,
)
best = runs[0]
# Then download plot_data artifact for curve reconstruction:
artifact_files = client.list_artifacts(run_id, path="plot_data")
local_dir = client.download_artifacts(run_id, artifact_files[0].path, tmpdir)
plot_df = pd.read_csv(local_dir)
```

11. Conclusiones

El deliverable minimo exigido por el profesor — un modelo entrenado y funcionando con tracking de experimentos — esta completamente cumplido por el pipeline local:

- dvc repro ejecuta las 4 etapas (pull_raw -> preprocess -> train -> evaluate) de forma reproducible y versionada.
- Los 7 runs quedan registrados en MLflow con parametros, metricas y artefactos.
- El mejor modelo (XGBoost, PR-AUC=0.8819) esta registrado con alias 'staging' y es cargable y funcional via `mlflow.pyfunc.load_model`.

El storage en nube via Cloudflare R2 esta listo arquitectonicamente: el bucket `luci-mlops-fraud` esta creado, la configuracion de DVC remote y `MLFLOW_S3_ENDPOINT_URL` esta documentada, y el unico paso pendiente es generar el token S3 permanente desde el dashboard de R2 y exportarlo como variable de entorno. Una vez hecho esto, dvc push y el artifact store de MLflow quedan operativos sin ningun cambio de codigo.

La pipeline es completamente reproducible: un clon del repositorio con `dvc pull` (cuando R2 este wired) seguido de `dvc repro` reconstruye metricas identicas, ya que `params.yaml`, `dvc.lock` y el hash del dataset estan versionados en Git.

Aprendizajes clave del proyecto:

- PR-AUC no es opcional cuando el dataset es tan desbalanceado. El colapso de `lgbm-l31` y `lgbm-l63` (ROC-AUC=0.89 pero PR-AUC=0.01) es la evidencia mas convincente.
- El tracking por run individual en MLflow permite diagnosticar modos de falla que son invisibles en promedios agregados.
- La API de alias de MLflow 3 es superior a los stages: mas flexible, mas expresiva y preparada para despliegues multi-canario.
- DVC + `params.yaml` crea un puente auditorio entre el versionado de datos/configuracion y el tracking de experimentos: toda la cadena es trazable desde el raw data hasta el modelo registrado.

12. Referencias

MLflow Model Registry — Aliases API

<https://mlflow.org/docs/latest/model-registry.html#model-aliases>

MLflow Tracking — log_metric, log_param

https://mlflow.org/docs/latest/python_api/mlflow.html

DVC Pipelines — dvc.yaml reference

<https://dvc.org/doc/user-guide/project-structure/dvcyaml-files>

DVC Remote Storage — S3 configuration

<https://dvc.org/doc/user-guide/data-management/remote-storage/amazon-s3>

Cloudflare R2 — S3 API compatibility

<https://developers.cloudflare.com/r2/api/s3/api/>

Cloudflare R2 — Tokens y autenticación

<https://developers.cloudflare.com/r2/api/s3/tokens/>

OpenML Dataset 1597 — Credit Card Fraud Detection

<https://www.openml.org/search?type=data&id=1597>

XGBoost — scale_pos_weight para datasets desbalanceados

https://xgboost.readthedocs.io/en/stable/tutorials/param_tuning.html

LightGBM — is_unbalance vs scale_pos_weight

https://lightgbm.readthedocs.io/en/latest/Parameters.html#is_unbalance